IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

INSTRUCTION ARCHITECTURES AND COMPILERS

Wednesday, 6 May 2026, 2:30 PM

Time allowed: 2:00 hours

There are NINE questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : P.Y.K. Cheung, J. Wickerson
Second Marker(s) : J. Wickerson, P.Y.K. Cheung

© Imperial College London

ELEC50010 INSTRUCTION ARCHITECTURES AND COMPILERS
MAY 2026

NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–9 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.

1. Using the instruction summary provided in the Appendix, translate the following RISC-V RV32I instructions either from assembly language to machine code or vice versa.

   a) `sw t0, 256(s10)` [3]

   b) `32'h02090493` [3]

2. The current values of the RISC-V processor's programme counter PC and register s1 are respectively:

   PC = 32'h1000; s1 = 32'h400

   What is the value of the register s0 after the following individual single instruction is executed separately:

   a) `auipc s0, 20'hA0001` [4]

   b) `slti s0, s1, 12'h800` [4]

3. The Branch (B-type) and Jump (J-type) instruction formats are shown in Figure 3.1.

   a) Briefly describe how the immediate value is computed by the assembler for all the branch instructions? [4]

   b) Briefly describe the operation of the instruction `jalr t0, t1, 31`. [4]

   c) Design in SystemVerilog the module *imm ()* as shown in Figure 3.2, which takes the 32-bit instruction code for B-type and J-type instructions and produces as output the 32-bit immediate operand required depending on the value of the input control signal B-type (true for Branch instructions). [6]

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]											[11]	imm[19:12]							rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4

Figure 3.1

```
module imm(
    input logic [31:0]  instr,      // 32-bit machine code instruction
    input logic         B_type,     // 1 for B-type, 0 for J-type
    output logic [31:0] imm_op      // 32-bit immediate operand
);
```

Figure 3.2

4. Figure 4.1 shows the state diagram of a finite state machine (FSM) that implements the 4-state branch prediction algorithm. Design in SystemVerilog the FSM using the module interface shown in Figure 4.2. [12]

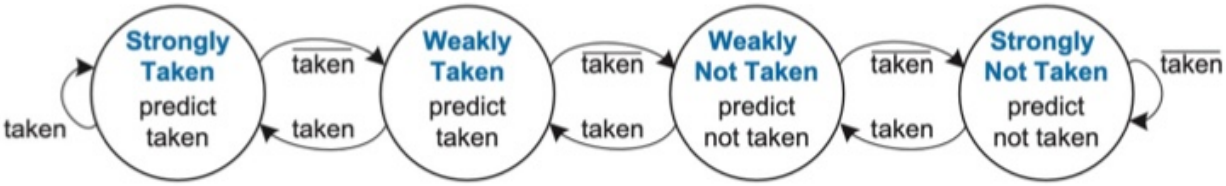


Figure 4.1

```
module branch_prediction_FSM(
    input  logic clk,            // system clock signal
    input  logic reset,          // FSM resert
    input  logic branch_taken    // branch resolved as taken
    output logic predict_taken   // 32-bit immediate operand
);
```

Figure 4.2

5. a) Explain briefly how principles of temporal and spatial locality enable cache memories to speed up the execution of software in computers. [4]

   b) A processor with data cache is used to execute a program that has 10,000 load operations from data memory. 8123 data access can be found in the cache memory and the rest are in other level of memory hierarchy. Calculate the cache hit and miss rates. Given that cache access takes 1 clock cycle while all other memory accesses require 100 clock cycles, calculate the total expected memory access time of the program. [6]

6. Explain how `continue` statements are handled by a C compiler. *[Hint: Note that* `continue` *statements may be compiled differently depending on where they appear in the source code.]* [6]

7. Here is an *ambiguous* context-free grammar:

$$X ::= \texttt{a} \mid X\texttt{b}X$$

a) Demonstrate that this grammar is ambiguous by drawing two different parse trees for the word `ababa`. [5]

b) Is X *nullable*? Briefly explain your answer. [2]

c) What is the *FIRST* set of X? Briefly explain your answer. [2]

d) What is the *FOLLOW* set of X? Briefly explain your answer. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [3]

e) Write down another context-free grammar that accepts the *same language*, but is no longer ambiguous. [4]

8. Gregory wants to make use of some library code in his program. He must decide whether to *statically* or to *dynamically* link his program with the library.

a) Explain one mechanism by which statically linking the library could make Gregory's program *more likely to crash*. [2]

b) Explain one mechanism by which statically linking the library could make Gregory's program *less likely to crash*. [2]

c) Explain one mechanism by which statically linking the library could make Gregory's program *faster to start up*. [2]

d) Explain one mechanism by which statically linking the library could make Gregory's program *slower to start up*. [2]

9. Consider the following C function:

```
int foo (int a) {
  int b, c, d; // 1
  b = a + 4;   // 2
  c = b - a;   // 3
  d = b + c;   // 4
  a = b + c;   // 5
  return a;
}
```

a) Annotate `foo` with the set of variables that are *live* at the end of each line (the positions labelled `1`–`5` in the code). [5]

b) Give two benefits of the *dead-code elimination* optimisation. [4]

c) Explain how the analysis carried out in part (a) enables dead-code elimination to be performed on `foo`. [2]

d) Now optimise `foo` *as much as possible*, and write down 32-bit RISC-V assembly code that implements it. Explain why your assembly code correctly implements `foo`. [9]

RISC-V Instruction Set Summary (1)

Register (R-type) Instructions

Bit position	31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
R-type	funct7	rs2	rs1	funct3	rd	opcode

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
7'h33	3'b010	7'h00	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
7'h33	3'b011	7'h00	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)^{unsigned}
7'h33	3'b100	7'h00	xor rd, rs1, rs2	bit-wise XOR	rd = rs1 ^ rs2
7'h33	3'b101	7'h00	srl rd, rs1, rs2	shfit right logical	rd = rs1 >> $rs2_{4:0}$
7'h33	3'b101	7'h20	sra rd, rs1, rs2	shfit right arithmetic	rd = rs1 >>> $rs2_{4:0}$
7'h33	3'b110	7'h00	or rd, rs1, rs2	bit-wise OR	rd = rs1 | rs2
7'h33	3'b111	7'h00	and rd, rs1, rs2	bit-wise AND	rd = rs1 & rs2

Immediate (I-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
I-type	imm[11:0]	rs1	funct3	rd	opcode

opcode	funct3	imm[11:5]	instruction	description	operation
7'h03	3'b000	-	`lb rd, imm(rs1)`	load byte immediate	rd = sign_ext ($[address]_{7:0}$)
7'h03	3'b001	-	`lh rd, imm(rs1)`	load half immediate	rd = sign_ext ($[address]_{15:0}$)
7'h03	3'b010	-	`lw rd, imm(rs1)`	load word immediate	rd = $[address]_{31:0}$
7'h03	3'b100	-	`lbu rd, imm(rs1)`	load byte unsigned	rd = zerp_ext ($[address]_{7:0}$)
7'h03	3'b101	-	`lhu rd, imm(rs1)`	load half unsigned	rd = zero_ext ($[address]_{15:0}$)
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b001	7'h00	`slli rd, rs1, uimm`	Shift left logical immediate	rd = rs1 << uimm
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))
7'h13	3'b011	-	`sltiu rd, rs1, uimm`	set less than imm. unsigned	rd = (rs1 < zero_ext(imm))
7'h13	3'b100	-	`xori rd, rs1, imm`	bit-wise XOR immediate	rd = rs1 ^ sign_ext (imm)
7'h13	3'b101	7'h00	`srli rd, rs1, uimm`	Shift right logical immediate	rd = rs1 >> uimm
7'h13	3'b101	7'h20	`srai rd, rs1, uimm`	Shift right arithmetic imm.	rd = rs1 >> uimm
7'h13	3'b110	-	`ori rd, rs1, imm`	bit-wise OR immediate	rd = rs1 | sign_ext (imm)
7'h13	3'b111	-	`andi rd, rs1, imm`	bit-wise AND immediate	rd = rs1 & sign_ext (imm)

Upper Immediate (U-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
U-type	imm[31:12]	rd	opcode

opcode	instruction	description	operation
7'h17	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
7'h37	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}

RISC-V Instruction Set Summary (2)

Store (S-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S-type	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						

opcode	funct3	instruction	description	operation
7'h23	3'b000	`sb rs2, imm(rs1)`	store byte immediate	$[address]_{7:0} = rs2_{7:0}$
7'h23	3'b001	`sh rs2, imm(rs1)`	store half immediate	$[address]_{15:0} = rs2_{15:0}$
7'h23	3'b010	`sw rs2, imm(rs1)`	store word immediate	$[address]_{31:0} = rs2$

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA
7'h63	3'b001	`bne rs1, rs2, label`	branch if not equal	If (rs1 ≠ rs2) PC = BTA
7'h63	3'b100	`blt rs1, rs2, label`	branch if less than	If (rs1 < rs2) PC = BTA
7'h63	3'b101	`bge rs1, rs2, label`	branch if greater than or equal	If (rs1 ≥ rs2) PC = BTA
7'h63	3'b110	`bltu rs1, rs2, label`	branch if less than unsigned	If (rs1 < rs2) PC = BTA
7'h63	3'b111	`bgeu rs1, rs2, label`	branch if greater or equal unsigned	If (rs1 ≥ rs2) PC = BTA

BTA = Branch Target Address, PC = Program Counter

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]										[11]	imm[19:12]								rd					opcode						
jalr – I-type	imm[11:0]												rs1					3'b000			rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4
7'h6f	`jal rd, label`	Jump and link	PC = JTA, rd = PC + 4

JTA = Jump Target Address, PC = Program Counter

Register names

name	register number	usage
zero	x0	Constant value 0
ra	x1	Return Addrss
sp	x2	Stack Pointer
gp	x3	Global Pointer
tp	X4	Thread Pointer
t0 – t2	x5 – x7	Temporary registers
s0/fp	x8	Saved register/Frame Pointer
s1	x9	Saved register
a0 – a1	x10 – x11	Function arguments/ return values
a2 – a7	x12 – x17	Function arguments
s2 – s11	x18 – x27	Saved registers
t3 – t6	x28 – x31	Temporary registers

Abbreviations

imm	12-bit signed immediate ($imm_{11:0}$)
uimm	5-bit unsigned immediate ($imm_{4:0}$)
upimm	20-bit upper immediate ($imm_{31:12}$)
address	memory addres: rs1 + sign_ext($imm_{11:0}$)
[address]	data at memory location address
BTA	branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0})
JTA	jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0})
label	text label indicating instruction address
sign_ext	value sign-extended to 32 bits
zero_ext	value zero-extended to 32 bits